CAPSTONE FINAL REPORT

(Sprint – 1)

Project Title : Fracto - Doctor Appointment Booking System

Student Name : Nikhil Kumar Sahu

Date : 16 March 2026

Instructor Details : Mr. Ramesh Nediyadath

Batch Name : WIPRO NGA - .Net Full Stack Angular - FY26 - C4

Table of Contents :	
Problem Definition and Objectives	
Frontend & Backend Architecture	
Database Design & Storage Optimization	
Admin Login & CRUD Operations	
User Registration & Login Operations	
Angular Front-End Template & Structure	
Code Sanitization - Components, Services & Modules	
CRUD Functionalities - Admin & User	
Advanced Features -Ratings & Real-Time Notifications	
Responsiveness & Client-Side Validation	
Session Handling	
Angular–Backend Integration & JWT Security	
Testing -Component & End-to-End	
GitHub Repository & Version Control	

1. Problem Definition and Objectives:

In India, patients face significant friction when trying to book appointments with specialist doctors. The process is manual, phone-based, and error-prone leading to double bookings, missed appointments, and no visibility into appointment status.

Objectives:

- Build a **full-stack web application** that allows patients to browse doctors, view specializations, and book time-slot-based appointments online.
- Provide an **Admin panel** with complete control over doctors, users, appointments, and specializations.
- Implement secure authentication using JWT tokens and salted password hashing (via ASP.NET Core Identity IPasswordHasher) so that users and admins have role-based access control.
- Deliver **real-time notifications** to users (appointment approved/cancelled) and admins (new booking) via SignalR WebSockets.
- Enable users to **rate and review** doctors after completing an appointment.

2. Frontend & Backend Architecture

Technology Stack

Layer	Technology
Frontend	Angular 18 (standalone components, signals)
Backend	ASP.NET Core 8 Web API
Database	Microsoft SQL Server (via EF Core 8)
Authentication	JWT Bearer Tokens
Real-Time	SignalR (WebSockets)
ORM	Entity Framework Core 8 (Code-First)
Styling	**Styling**: Tailwind CSS (utility-first), responsive layout using **flexbox** and **CSS grid**

System Architecture Diagram

```
+-----------+          HTTPS / REST / SignalR          +------------------+
|  Angular  |  ------------------------------------>   | ASP.NET Core API |
|  (Port    |  <------------------------------------   | (Port 5186)      |
|   4200)   |                                          |                  |
+-----------+                                          |  Controllers     |
                                                       |  JWT Middleware  |
                                                       |  SignalR Hub     |
                                                       +--------+---------+
                                                                |
                                                       EF Core (Code-First)
                                                                |
                                                       +--------+---------+
                                                       |   SQL Server     |
                                                       |   (FractoDb)     |
                                                       +------------------+
```

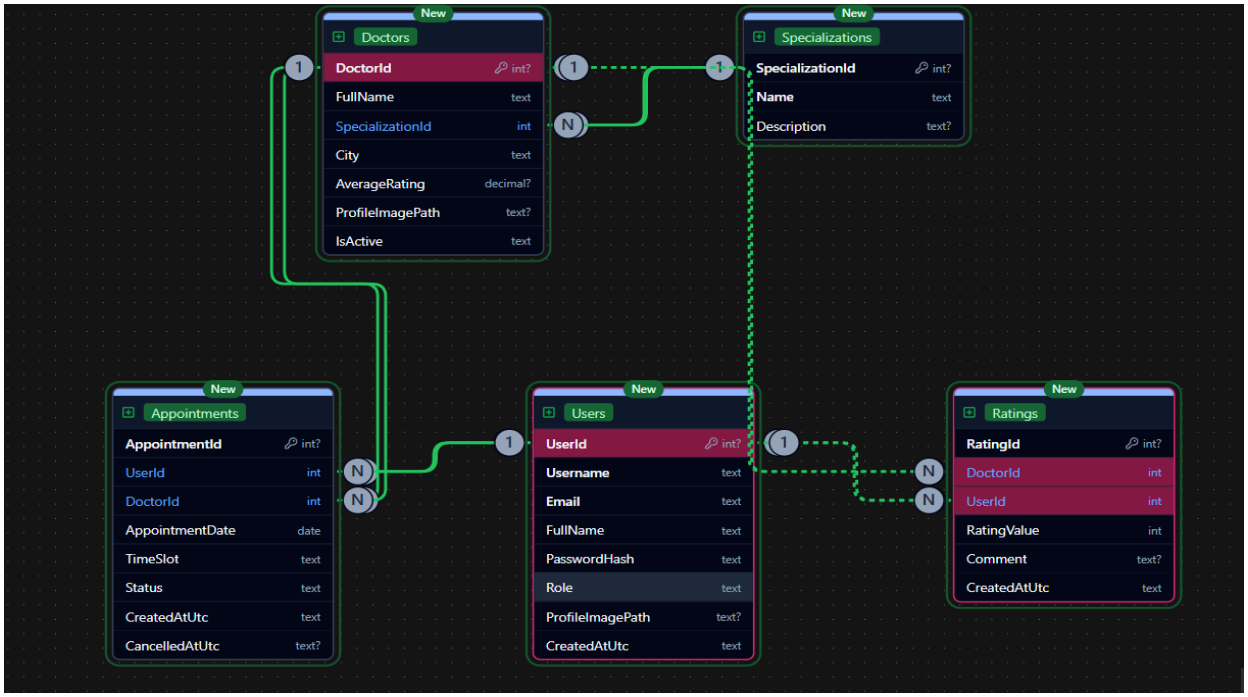
API Overview

The backend exposes the following route groups:

Route Prefix	**Purpose**	**Auth Required**
POST /api/auth/register	User Registration	No
POST /api/auth/login	User Login	No
GET /api/auth/me	Get Current user profile	JWT
GET /api/doctors	Browse all doctors	No
GET /api/doctors/{id}/available-slots	Get available time slots	No
GET /api/appointments/my	My appointments	JWT (User)
POST /api/appointments	Book appointment	JWT (User)
DELETE /api/appointments/{id}	Cancel appointment	JWT (User)
GET /api/ratings	Doctor ratings	No

POST /api/ratings	Submit a rating	JWT (User)
GET /api/admin/stats	Dashboard stats	JWT (Admin)
GET/POST/PUT/DELETE /api/admin/users	Manage users	JWT (Admin)
GET/POST/PUT/DELETE /api/admin/doctors	Manage doctors	JWT (Admin)
GET /api/admin/appointments	All appointments	JWT (Admin)
PUT /api/admin/appointments/{id}/approve	Approve appointment	JWT (Admin)
PUT /api/admin/appointments/{id}/complete	Mark Complete	JWT (Admin)
PUT /api/admin/appointments/{id}/cancel	Cancel appointment	JWT (Admin)
/hubs/notifications	Real-time SignalR hub	JWT

3. Database Design & Storage Optimization



<u>Tables and Relationships</u>

Users Table

- Primary Key: Id (auto-increment int)

- Stores: username, email, full name, bcrypt-hashed password, role (enum: User=1, Admin=2), optional profile image path, and UTC creation timestamp
- Relationships: One user has many Appointments and many Ratings

Doctors Table

- Primary Key: Id (auto-increment int)
- Foreign Key: SpecializationId → Specializations.Id
- Stores: full name, city, computed average rating, optional profile image, and an IsActive flag for soft-deactivation
- Relationships: One doctor belongs to one Specialization; has many Appointments and Ratings

Specializations Table

- Primary Key: Id (auto-increment int)
- Stores: name (e.g., "Cardiologist") and description
- Relationship: One specialization has many Doctors

Appointments Table

- Primary Key: Id (auto-increment int)
- Foreign Keys: UserId → Users.Id; DoctorId → Doctors.Id
- Stores: appointment date (DateOnly), time slot string (e.g., "10:00–10:30"), status enum (Booked=1/Approved=2/Completed=3/Cancelled=4), created and cancelled timestamps

Ratings Table

- Primary Key: Id (auto-increment int)
- Foreign Keys: UserId → Users.Id; DoctorId → Doctors.Id
- Stores: rating value (1–5), optional comment, timestamp
- Business Rule: A user can only rate a doctor after having a non-cancelled appointment

Code - Entity Models

1.User.cs

```
public enum UserRole { User = 1, Admin = 2 }

public class User
{
    public int Id { get; set; }

    [Required, MaxLength(50)]
    public string Username { get; set; } = string.Empty;

    [Required, MaxLength(150), EmailAddress]
    public string Email { get; set; } = string.Empty;

    [Required, MaxLength(100)]
    public string FullName { get; set; } = string.Empty;

    [Required]
    public string PasswordHash { get; set; } = string.Empty;

    public UserRole Role { get; set; } = UserRole.User;

    [MaxLength(300)]
    public string? ProfileImagePath { get; set; }

    public DateTime CreatedAtUtc { get; set; } = DateTime.UtcNow;

    // Navigation properties (EF Core relationships)
    public ICollection<Appointment> Appointments { get; set; } = new List<Appointment>();
    public ICollection<Rating> Ratings { get; set; } = new List<Rating>();
}
```

2.Doctor.cs

```
public class Doctor
{
    public int Id { get; set; }
    [Required, MaxLength(100)]
    public string FullName { get; set; } = string.Empty;

    public int SpecializationId { get; set; }
    public Specialization? Specialization { get; set; }  // Navigation

    [Required, MaxLength(100)]
    public string City { get; set; } = string.Empty;

    public decimal? AverageRating { get; set; }

    [MaxLength(300)]
    public string? ProfileImagePath { get; set; }

    public bool IsActive { get; set; } = true;

    public ICollection<Appointment> Appointments { get; set; } = new List<Appointment>();
    public ICollection<Rating> Ratings { get; set; } = new List<Rating>();
}
```

3.Appointment.cs

```csharp
public enum AppointmentStatus
{
    Booked = 1, Approved = 2, Completed = 3, Cancelled = 4
}

public class Appointment
{
    public int Id { get; set; }
    public int UserId { get; set; }
    public User? User { get; set; }
    public int DoctorId { get; set; }
    public Doctor? Doctor { get; set; }
    [Required]
    public DateOnly AppointmentDate { get; set; }
    [Required, MaxLength(20)]
    public string TimeSlot { get; set; } = string.Empty; // e.g. "10:00-10:30"
    public AppointmentStatus Status { get; set; } = AppointmentStatus.Booked;
    public DateTime CreatedAtUtc { get; set; } = DateTime.UtcNow;
    public DateTime? CancelledAtUtc { get; set; }
}
```

Database Seeding (Program.cs)

On application startup, the database is automatically seeded with:

- **Default Admin**: username nikhil11012, email nikhil11012@gmail.com, password hashed using ASP.NET Identity PasswordHasher
- **Specializations**: Cardiologist, Dermatologist, Neurologist, Orthopedic etc.
- **Sample Doctors**: Dr. A. Sharma (Delhi), Dr. B. Verma (Mumbai), Dr. C. Gupta (Delhi), Dr. D. Iyer (Bengaluru), Dr. E. Khan (Kolkata) etc.

Optimization Techniques

- AsNoTracking() is used on all read-only queries to avoid EF Core change tracking overhead.
- Selective column projections (.Select(u => new { u.Id, u.Username, ... })) avoid fetching the PasswordHash column in admin user listings.
- Eager loading (Include) is used only when navigation properties are needed (e.g., doctor's specialization name in appointment listings).

- AnyAsync() is used for existence checks (duplicate username/email, slot conflict) instead of fetching full entities.

4. Admin Login & CRUD Operations

Admin Authentication

The admin account is seeded at startup. Admin login uses the same /api/auth/login endpoint. The difference is in the JWT token. when an admin logs in, their token contains ClaimTypes.Role = "Admin". All admin controllers are decorated with [Authorize(Roles = "Admin")], so the backend enforces role-based access automatically.

Code - AuthController (Login & Register)

```
[ApiController]
[Route("api/[controller]")]
public class AuthController : ControllerBase
{
    private readonly AppDbContext _db;
    private readonly JwtOptions _jwt;
    private readonly IPasswordHasher<User> _passwordHasher;

    public AuthController(
        AppDbContext db,
        IOptions<JwtOptions> jwtOptions,
        IPasswordHasher<User> passwordHasher)
    {
        _db = db;
        _jwt = jwtOptions.Value;
        _passwordHasher = passwordHasher;
    }

    [HttpPost("login")]
    public async Task<ActionResult<AuthResponse>> Login(LoginRequest request)
    {
        var identifier = request.Identifier.Trim();
        var identifierLower = identifier.ToLowerInvariant();

        // Supports login via either username OR email
        var user = await _db.Users.SingleOrDefaultAsync(x =>
            x.Username == identifier || x.Email == identifierLower);

        if (user is null)
            return Unauthorized(new { message = "Invalid credentials." });

        var verify = _passwordHasher.VerifyHashedPassword(
            user, user.PasswordHash, request.Password);

        if (verify == PasswordVerificationResult.Failed)
            return Unauthorized(new { message = "Invalid credentials." });

        return Ok(CreateAuthResponse(user));  // Returns JWT token
    }
}
```

Explanation: The login method accepts either a username or email as the identifier, trims whitespace, checks if the user exists, then uses ASP.NET

Identity's PasswordHasher.VerifyHashedPassword() to safely compare the hashed password, no plain-text comparison ever occurs.

Code - Admin Dashboard Stats

```
[Authorize(Roles = "Admin")]
[HttpGet("stats")]
public async Task<IActionResult> GetStats()
{
    var totalUsers        = await _db.Users.CountAsync();
    var totalDoctors      = await _db.Doctors.CountAsync();
    var totalAppointments      = await _db.Appointments.CountAsync();
    var pendingAppointments    = await _db.Appointments
        .CountAsync(a => a.Status == AppointmentStatus.Booked);

    return Ok(new {
        TotalUsers = totalUsers,
        TotalDoctors = totalDoctors,
        TotalAppointments = totalAppointments,
        PendingAppointments = pendingAppointments
    });
}
```

5. User Registration & Login Operations

Registration Flow

1. User submits { username, email, fullName, password } to POST /api/auth/register
2. Backend checks if username or email already exists (returns 400 Bad Request if so)
3. Password is hashed using IPasswordHasher<User> (BCrypt-based, salted)
4. User is saved to the database with Role = UserRole.User
5. A JWT token is immediately returned, logging the user in right after registration

Code – Register Endpoint

```
[HttpPost("register")]
public async Task<ActionResult<AuthResponse>> Register(RegisterRequest request)
{
    var username = request.Username.Trim();
    var email    = request.Email.Trim().ToLowerInvariant();

    // Duplicate checks
    var usernameExists = await _db.Users.AnyAsync(x => x.Username == username);
    if (usernameExists)
        return BadRequest(new { message = "Username already exists." });

    var emailExists = await _db.Users.AnyAsync(x => x.Email == email);
    if (emailExists)
        return BadRequest(new { message = "Email already exists." });

    var user = new User
    {
        Username = username,
        Email    = email,
        FullName = request.FullName.Trim(),
        Role     = UserRole.User  // Regular user by default
    };

    // Hash password BEFORE saving
    user.PasswordHash = _passwordHasher.HashPassword(user, request.Password);

    _db.Users.Add(user);
    await _db.SaveChangesAsync();

    // Auto-login after registration
    return Ok(CreateAuthResponse(user));
}
```

JWT Token Generation

```
private AuthResponse CreateAuthResponse(User user)
{
    var now     = DateTime.UtcNow;
    var expires = now.AddMinutes(_jwt.ExpiryMinutes); // 60 minutes

    var claims = new List<Claim>
    {
        new(ClaimTypes.NameIdentifier, user.Id.ToString()),
        new(ClaimTypes.Name,           user.FullName),
        new(ClaimTypes.Email,          user.Email),
        new(ClaimTypes.Role,           user.Role.ToString()) // "User" or "Admin"
    };

    var key   = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_jwt.Key));
    var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);

    var token = new JwtSecurityToken(
        issuer:            _jwt.Issuer,
        audience:          _jwt.Audience,
        claims:            claims,
        notBefore:         now,
        expires:           expires,
        signingCredentials: creds
    );

    return new AuthResponse
    {
        Token           = new JwtSecurityTokenHandler().WriteToken(token),
        ExpiresAtUtc    = expires,
        UserId          = user.Id,
        Username        = user.Username,
        Email           = user.Email,
        Role            = user.Role,
        ProfileImagePath= user.ProfileImagePath
    };
}
```

Explanation: The JWT contains four claims: the user's database ID (used to look up ownership for appointments/ratings), full name, email, and role. The role claim is what ASP.NET Core uses to enforce [Authorize(Roles = "Admin")] on protected endpoints. The token is signed with HMAC-SHA256, making it tamper-evident.

6. Angular Front - End Template & Structure

```
Fracto.Web/
├── src/
│   ├── app/
│   │   ├── app.ts                  ← Root component (shell, nav-bar)
│   │   ├── app.html                ← Root template with router-outlet
│   │   ├── app.routes.ts           ← Centralized lazy-loaded routing
│   │   ├── app.config.ts           ← App-level providers (router, HTTP, interceptors)
│   │   ├── core/
│   │   │   ├── services/
│   │   │   │   ├── auth.service.ts          ← Login, register, session management
│   │   │   │   ├── admin.service.ts         ← All admin API calls
│   │   │   │   ├── appointment.service.ts   ← Booking, listing, cancelling
│   │   │   │   ├── doctor.service.ts        ← Doctor listing & detail
│   │   │   │   ├── rating.service.ts        ← Submit & fetch ratings
│   │   │   │   └── notification.service.ts  ← SignalR connection management
│   │   │   ├── guards/
│   │   │   │   └── auth.guard.ts            ← authGuard & adminGuard
│   │   │   └── interceptors/
│   │   │       └── auth.interceptor.ts      ← Auto-attach JWT to HTTP requests
│   │   └── features/
│   │       ├── home/                        ← Doctor listing page
│   │       ├── auth/
│   │       │   ├── login/                   ← Login form
│   │       │   └── register/                ← Registration form
│   │       ├── doctors/
│   │       │   ├── doctor-details/          ← Doctor profile & booking
│   │       │   └── booking-confirmation/    ← Booking success screen
│   │       ├── dashboard/
│   │       │   └── my-appointments/         ← User's appointment list
│   │       └── admin/
│   │           └── admin-dashboard/         ← Full admin panel
│   ├── environments/
│   │   └── environment.ts          ← API URL config
│   └── index.html                  ← Single HTML entry point
```

Core Routing (app.routes.ts)

```
export const routes: Routes = [
  // Home — public doctor listing
  { path: '', loadComponent: () =>
      import('./features/home/home').then(m => m.Home) },

  // Doctor detail page — public
  { path: 'doctors/:id', loadComponent: () =>
      import('./features/doctors/doctor-details/doctor-details').then(m => m.DoctorDetails) },

  // Booking confirmation — public
  { path: 'booking-confirmation/:id', loadComponent: () =>
      import('./features/doctors/booking-confirmation/booking-confirmation').then(m => m.BookingConfirmation) },

  // My appointments — requires login
  { path: 'my-appointments', loadComponent: () =>
      import('./features/dashboard/my-appointments/my-appointments').then(m => m.MyAppointments),
    canActivate: [authGuard] },

  // Admin panel — requires admin role
  { path: 'admin', loadComponent: () =>
      import('./features/admin/admin-dashboard/admin-dashboard').then(m => m.AdminDashboard),
    canActivate: [adminGuard] },

  // Auth pages
  { path: 'login',    loadComponent: () =>
      import('./features/auth/login/login').then(m => m.Login) },
  { path: 'register', loadComponent: () =>
      import('./features/auth/register/register').then(m => m.Register) },
];
```

Explanation: All routes use Angular's loadComponent() for **lazy loading** - meaning each feature's code is only downloaded when the user navigates to that route. This significantly reduces the initial bundle size and improves startup performance.

App Config (app.config.ts)

```
export const appConfig: ApplicationConfig = {
  providers: [
    provideZoneChangeDetection({ eventCoalescing: true }),
    provideRouter(routes),
    provideHttpClient(withInterceptors([authInterceptor]))
    //                                  ↑
    //       Registers the JWT interceptor globally for all HTTP calls
  ]
};
```

7. Code Sanitization - Components, Services & Modules

Architecture Principles Followed

The project follows Angular's recommended **feature + core** folder pattern:

- **core/** — Application-wide singletons only (services, guards, interceptors). Never imports from features/.
- **features/** — Each feature (auth, admin, doctors, dashboard) is self-contained with its own component, template, and styles.
- **Standalone Components** — Angular 18's standalone API is used throughout; there are no NgModule declarations. Each component declares its own imports in the @Component decorator.

Services (core/services)

Service	Responsibility
AuthService	Login, register, logout, session restore, profile image upload
AdminService	All admin API calls: users, doctors, appointments, specializations, stats
AppointmentService	Book, list, cancel appointments; fetch available slots
DoctorService	List doctors with filters, fetch single doctor detail
RatingService	Submit and fetch doctor ratings
NotificationService	Manage the SignalR WebSocket connection and notification state

All services are providedIn: 'root' meaning they are application-wide singletons, created once and destroyed when the app closes.

Guards (core/guards/auth.guard.ts)

```
export const authGuard: CanActivateFn = (route, state) => {
  const authService = inject(AuthService);
  const router      = inject(Router);

  if (authService.isAuthenticated()) return true;

  // Redirect unauthenticated users to login
  return router.createUrlTree(['/login']);
};

// adminGuard — Protects admin-only pages
export const adminGuard: CanActivateFn = (route, state) => {
  const authService = inject(AuthService);
  const router      = inject(Router);

  if (authService.isAuthenticated() && authService.isAdmin())
    return true;

  // Authenticated non-admins go home; unauthenticated go to login
  return authService.isAuthenticated()
    ? router.createUrlTree(['/'])
    : router.createUrlTree(['/login']);
};
```

Explanation: Two guards are defined in one file: authGuard blocks unauthenticated users from protected routes (like /my-appointments), while adminGuard additionally checks the user's role (stored in sessionStorage) before allowing access to /admin.

Interceptor (core/interceptors/auth.interceptor.ts)

```
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const authService = inject(AuthService);
  const token       = authService.getToken();  // Reads JWT from sessionStorage

  if (token) {
    // Clone the request and add Authorization header
    const clonedReq = req.clone({
      headers: req.headers.set('Authorization', `Bearer ${token}`)
    });
    return next(clonedReq);
  }

  return next(req);  // No token — send request as-is (public endpoints)
};
```

Explanation: The HTTP interceptor intercepts every outgoing HTTP request. If a JWT token is present in session storage, it clones the request and adds the Authorization: Bearer <token> header before forwarding the request. This replaces the need to manually attach headers in every service method.

8. CRUD Functionalities - Admin & User

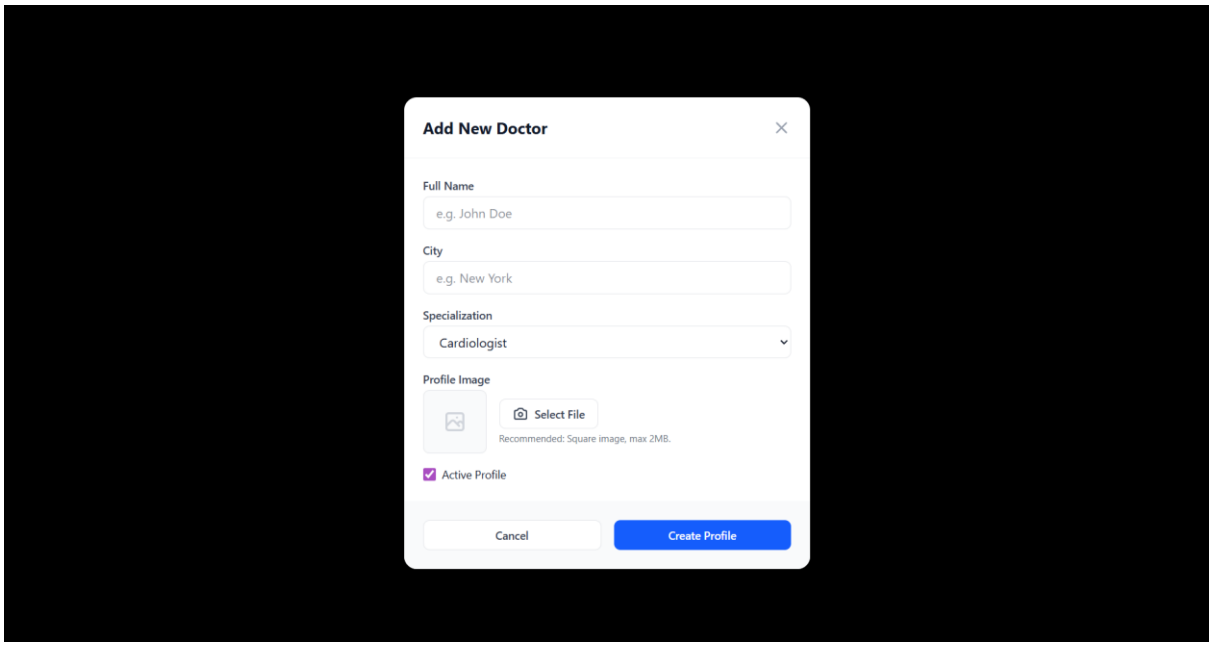
Admin: Doctor Management (Full CRUD)

```
// READ ALL
[HttpGet]
public async Task<ActionResult<List<DoctorListItemDto>>> GetAll()
{
    var items = await _db.Doctors
        .AsNoTracking()
        .Include(d => d.Specialization)       // Eager-load specialization name
        .OrderBy(d => d.FullName)
        .Select(d => new DoctorListItemDto { ... })
        .ToListAsync();
    return Ok(items);
}

// CREATE
[HttpPost]
public async Task<ActionResult<DoctorListItemDto>> Create([FromBody] DoctorUpsertDto dto)
{
    var doctor = new Doctor {
        FullName         = dto.FullName,
        City             = dto.City,
        SpecializationId = dto.SpecializationId,
        IsActive         = dto.IsActive,
        ProfileImagePath = dto.ProfileImagePath
    };
    _db.Doctors.Add(doctor);
    await _db.SaveChangesAsync();
    return await GetById(doctor.Id);  // Return full DTO with specialization name
}
```

```
[HttpPut("{id:int}")]
public async Task<IActionResult> Update(int id, [FromBody] DoctorUpsertDto dto)
{
    var doctor = await _db.Doctors.FindAsync(id);
    if (doctor is null) return NotFound();
    doctor.FullName         = dto.FullName;
    doctor.City             = dto.City;
    doctor.SpecializationId = dto.SpecializationId;
    doctor.IsActive         = dto.IsActive;
    doctor.ProfileImagePath = dto.ProfileImagePath;
    await _db.SaveChangesAsync();
    return NoContent();
}

// DELETE — with safety check
[HttpDelete("{id:int}")]
public async Task<IActionResult> Delete(int id)
{
    var doctor = await _db.Doctors.FindAsync(id);
    if (doctor is null) return NotFound();

    var hasAppointments = await _db.Appointments.AnyAsync(a => a.DoctorId == id);
    if (hasAppointments)
        return BadRequest(new { message = "Cannot delete doctor with existing appointments. Deactivate instead."
});

    _db.Doctors.Remove(doctor);
    await _db.SaveChangesAsync();
    return NoContent();
}
```

Admin: User Management (CRUD)

```
// READ ALL — password hash intentionally excluded from output
[HttpGet]
public async Task<ActionResult<List<object>>> GetAll()
{
    var users = await _db.Users
        .AsNoTracking()
        .OrderByDescending(u => u.CreatedAtUtc)
        .Select(u => new { u.Id, u.Username, u.Email, u.FullName, u.Role, u.ProfileImagePath, u.CreatedAtUtc })
        .ToListAsync();
    return Ok(users);
}

// UPDATE user (role, email, name)
[HttpPut("{id:int}")]
public async Task<IActionResult> Update(int id, [FromBody] UserUpdateDto dto)
{
    var user = await _db.Users.FindAsync(id);
    if (user is null) return NotFound();
    user.FullName = dto.FullName;
    user.Email    = dto.Email;
    user.Role     = dto.Role;
    await _db.SaveChangesAsync();
    return NoContent();
}

// DELETE user
[HttpDelete("{id:int}")]
public async Task<IActionResult> Delete(int id)
{
    var user = await _db.Users.FindAsync(id);
    if (user is null) return NotFound();
    _db.Users.Remove(user);
    await _db.SaveChangesAsync();
    return NoContent();
}
```



Admin: Appointment Management (Approve / Complete / Cancel)

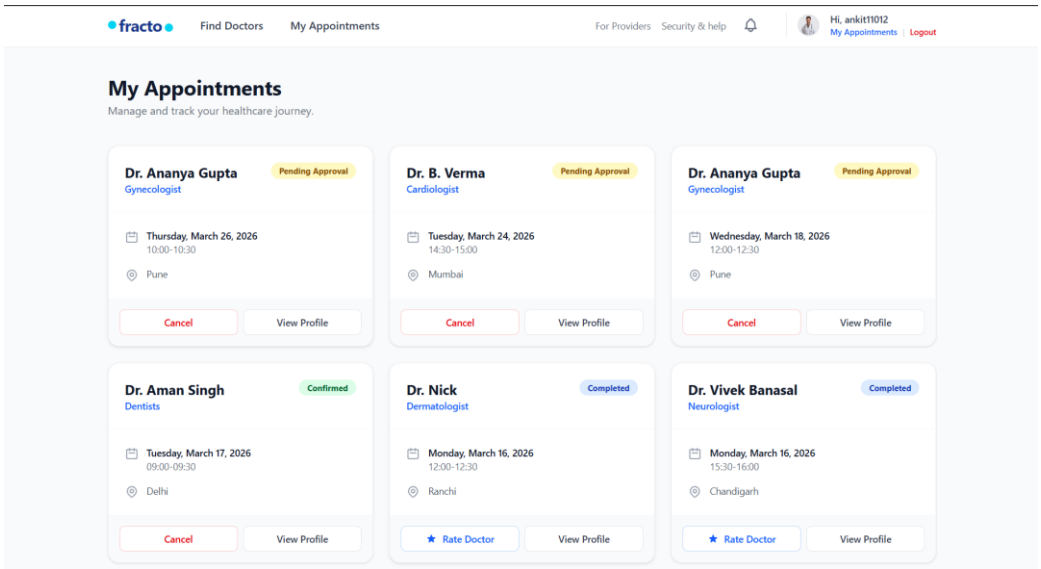
```
// Approve appointment + notify user via SignalR
[HttpPut("{id:int}/approve")]
public async Task<IActionResult> Approve(int id)
{
    var appointment = await _db.Appointments.FindAsync(id);
    if (appointment is null) return NotFound();

    if (appointment.Status == AppointmentStatus.Cancelled)
        return BadRequest(new { message = "Cannot approve a cancelled appointment." });

    appointment.Status = AppointmentStatus.Approved;
    await _db.SaveChangesAsync();

    // Real-time push to the specific user
    await _hubContext.Clients.User(appointment.UserId.ToString())
        .SendAsync("ReceiveNotification", "Your appointment has been approved! ✅");

    return NoContent();
}
```

Overview | User Management | Doctor Management | Global Appointments | Specializations

ID	PATIENT	DOCTOR	DATE/TIME	STATUS	ACTIONS
#14	Ankit Kumar ankit11012@gmail.com	Dr. Ananya Gupta Gynecologist	Mar 26, 2026 10:00-10:30	Pending	Approve Cancel
#15	Ankit Kumar ankit11012@gmail.com	Dr. B. Verma Cardiologist	Mar 24, 2026 14:30-15:00	Pending	Approve Cancel
#19	Ankit Kumar ankit11012@gmail.com	Dr. Ananya Gupta Gynecologist	Mar 18, 2026 12:00-12:30	Pending	Approve Cancel
#11	Ankit Kumar ankit11012@gmail.com	Dr. Aman Singh Dentists	Mar 17, 2026 09:00-09:30	Approved	Complete Cancel
#8	Ankit Kumar ankit11012@gmail.com	Dr. Nick Dermatologist	Mar 16, 2026 12:00-12:30	Completed	Cancel
#10	Aakash Kumar aakash11012@gmail.com	Dr. Nick Dermatologist	Mar 16, 2026 12:30-13:00	Approved	Complete Cancel
#18	Ankit Kumar ankit11012@gmail.com	Dr. Vivek Banasal Neurologist	Mar 16, 2026 15:30-16:00	Completed	Cancel
#9	Ankit Kumar ankit11012@gmail.com	Dr. Karan Malhotra Neurologist	Mar 15, 2026 12:00-12:30	Cancelled	
#16	Ankit Kumar ankit11012@gmail.com	Dr. A. Sharma Cardiologist	Mar 15, 2026 15:00-15:30	Completed	Cancel

User: Book & Cancel Appointments

```
// AppointmentsController.cs

[HttpPost]
public async Task<ActionResult<AppointmentDto>> BookAppointment(BookAppointmentRequest request)
{
    var userId = GetCurrentUserId();
    if (userId is null) return Unauthorized();

    var doctor = await _db.Doctors
        .Include(d => d.Specialization)
        .SingleOrDefaultAsync(d => d.Id == request.DoctorId);

    if (doctor is null || !doctor.IsActive)
        return BadRequest(new { message = "Doctor not found or inactive." });

    // DOUBLE BOOKING PREVENTION — checks same doctor + date + slot
    var exists = await _db.Appointments.AnyAsync(a =>
        a.DoctorId          == request.DoctorId &&
        a.AppointmentDate   == request.AppointmentDate &&
        a.TimeSlot          == request.TimeSlot.Trim() &&
        a.Status            != AppointmentStatus.Cancelled);  // Cancelled slots are freed

    if (exists) return Conflict(new { message = "This time slot is already booked." });

    var appointment = new Appointment { ... };
    _db.Appointments.Add(appointment);
    await _db.SaveChangesAsync();

    // Notify all admins about the new booking
    var patientName = User.Identity?.Name ?? "A patient";
    await _hubContext.Clients.Group("Admins")
        .SendAsync("ReceiveAdminNotification",
                   $"{patientName} has booked an appointment with Dr. {doctor.FullName}. 📅");

    return CreatedAtAction(nameof(GetMyAppointments), new { }, dto);
}
```



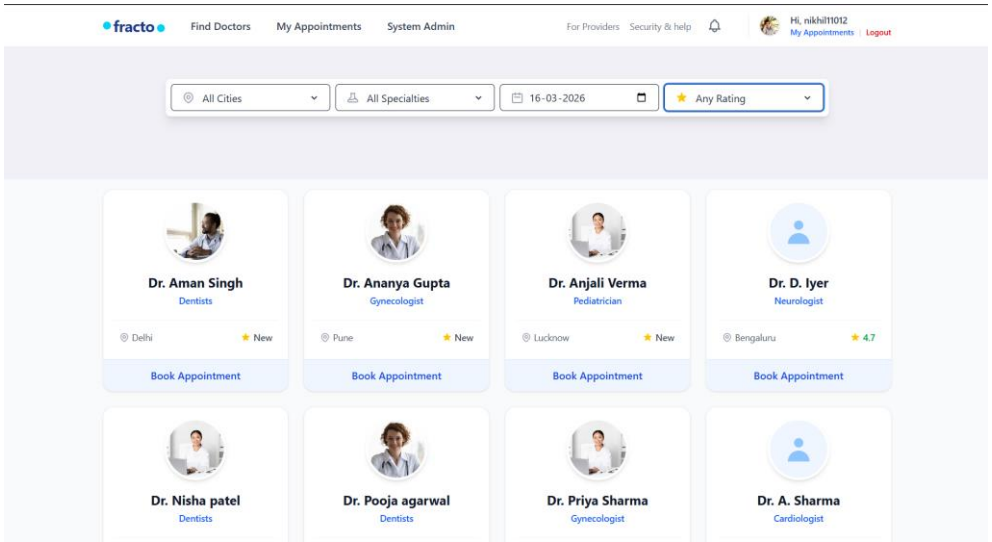
9. Advanced Features - Ratings & Real-Time Notifications

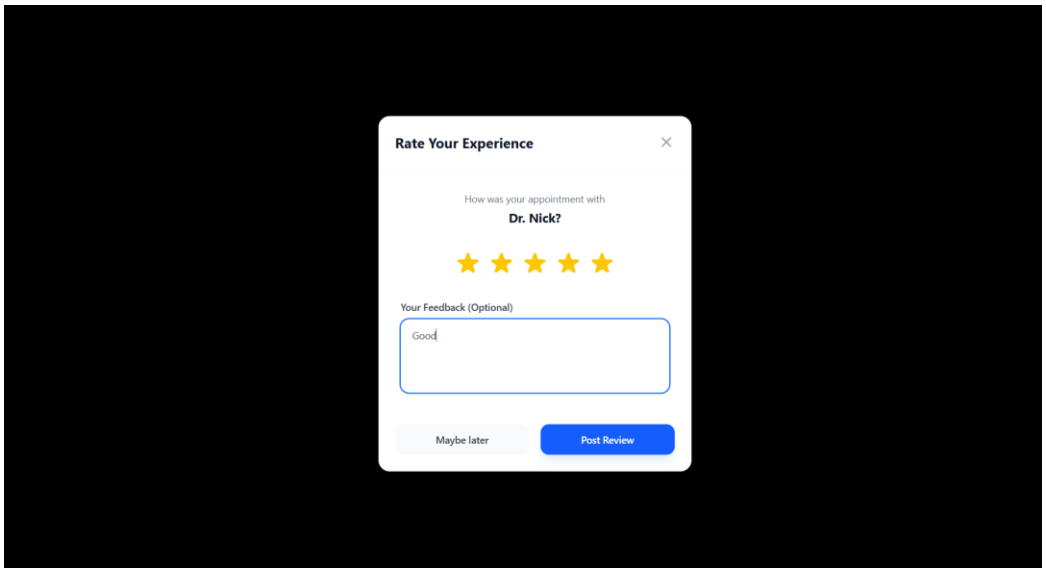
Feature 1: Doctor Ratings & Reviews

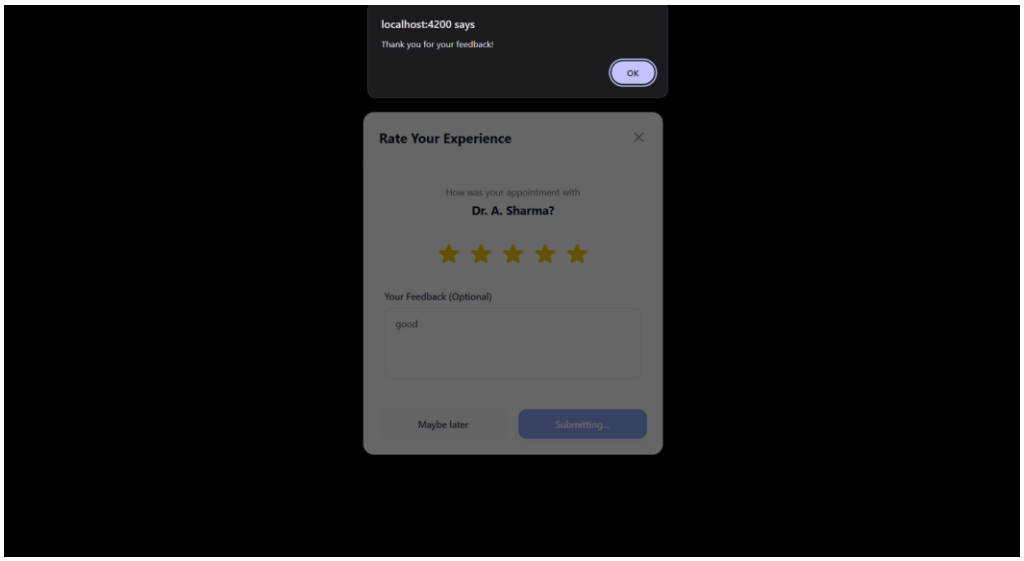
The ratings system enforces the business rule that a user can only rate a doctor after having a non-cancelled appointment with them.

```
// RatingsController.cs

[HttpPost]
public async Task<ActionResult<RatingDto>> CreateRating(CreateRatingRequest request)
{
    var userId = GetCurrentUserId();
    if (userId is null) return Unauthorized();

    // Rule: Must have at least one non-cancelled appointment with this doctor
    var hasAppointment = await _db.Appointments.AnyAsync(a =>
        a.UserId   == userId &&
        a.DoctorId == request.DoctorId &&
        a.Status   != AppointmentStatus.Cancelled);

    if (!hasAppointment)
        return BadRequest(new { message = "You can rate a doctor only after having an appointment." });

    // Upsert: update existing rating or create new one
    var existing = await _db.Ratings.SingleOrDefaultAsync(r =>
        r.DoctorId == request.DoctorId && r.UserId == userId);

    if (existing is null)
    {
        _db.Ratings.Add(new Rating {
            DoctorId    = request.DoctorId,
            UserId      = userId.Value,
            RatingValue = request.RatingValue,
            Comment     = request.Comment
        });
    }
    else
    {
        existing.RatingValue = request.RatingValue;
        existing.Comment     = request.Comment;
    }

    await _db.SaveChangesAsync();
```

```
// Recalculate and store the doctor's average rating
var allRatings        = await _db.Ratings
    .Where(r => r.DoctorId == request.DoctorId)
    .Select(r => r.RatingValue)
    .ToListAsync();

doctor.AverageRating = allRatings.Count > 0 ? (decimal)allRatings.Average() : null;
await _db.SaveChangesAsync();

return Ok(dto);
```


fracto
Find Doctors
My Appointments
System Admin
For Providers
Security & help
Hi, nikhil11012
My Appointments
Logout
All Cities
All Specialties
16-03-2026
Any Rating
Dr. Aman Singh
Dentists
Delhi
New
Book Appointment
Dr. Ananya Gupta
Gynecologist
Pune
New
Book Appointment
Dr. Anjali Verma
Pediatrician
Lucknow
New
Book Appointment
Dr. D. Iyer
Neurologist
Bengaluru
4.7
Book Appointment
Dr. Nisha patel
Dentists
Dr. Pooja agarwal
Dentists
Dr. Priya Sharma
Gynecologist
Dr. A. Sharma
Cardiologist


Rate Your Experience
How was your appointment with
Dr. Nick?
Your Feedback (Optional)
Good
Maybe later
Post Review

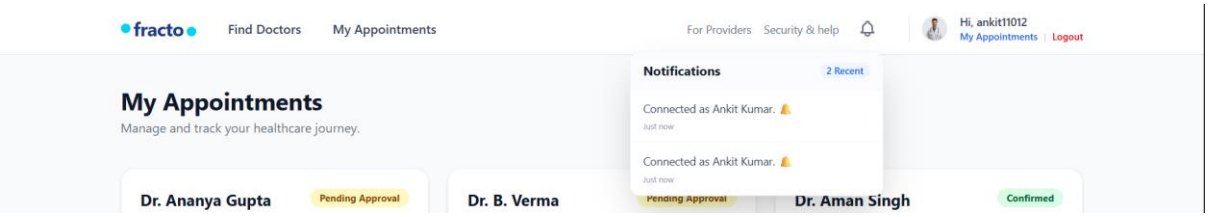

localhost:4200 says
Thank you for your feedback!
OK
Rate Your Experience
How was your appointment with
Dr. A. Sharma?
Your Feedback (Optional)
good
Maybe later
Submitting...

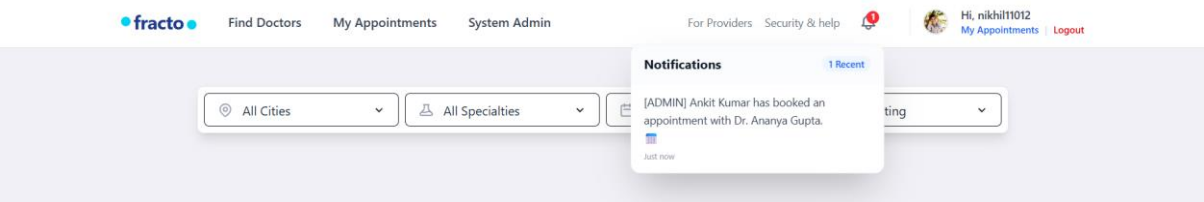
Feature 2 : Real-Time SignalR Notifications

SignalR provides a persistent WebSocket connection so users receive instant notifications without polling.

Backend Hub Configuration (Program.cs):

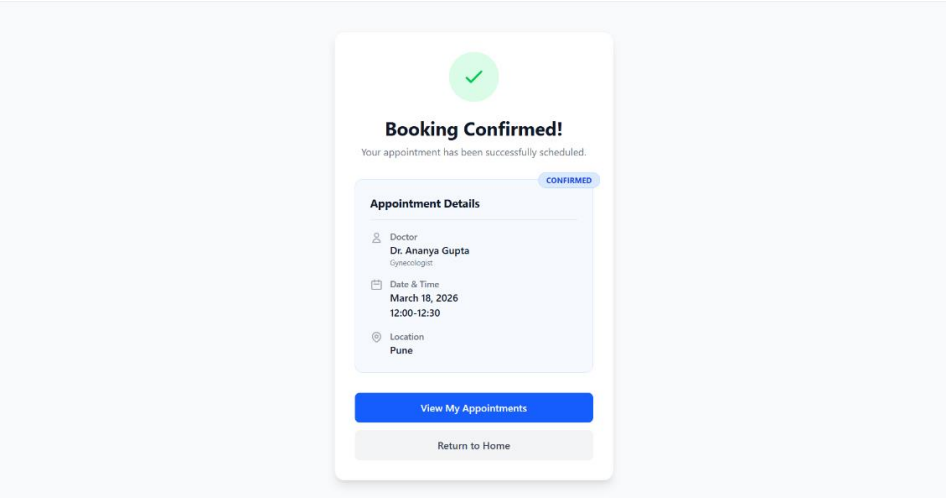
```
// Register SignalR service
builder.Services.AddSignalR();

// Map the hub to a URL
app.MapHub<Fracto.Api.Hubs.NotificationHub>("/hubs/notifications");

// JWT for SignalR: browsers can't set auth headers on WebSockets,
// so the token is read from the query string instead
options.Events = new JwtBearerEvents
{
    OnMessageReceived = context => {
        var accessToken = context.Request.Query["access_token"];
        var path        = context.HttpContext.Request.Path;
        if (!string.IsNullOrEmpty(accessToken) &&
            path.StartsWithSegments("/hubs/notifications"))
        {
            context.Token = accessToken;  // Use token from query param
        }
        return Task.CompletedTask;
    }
};
```

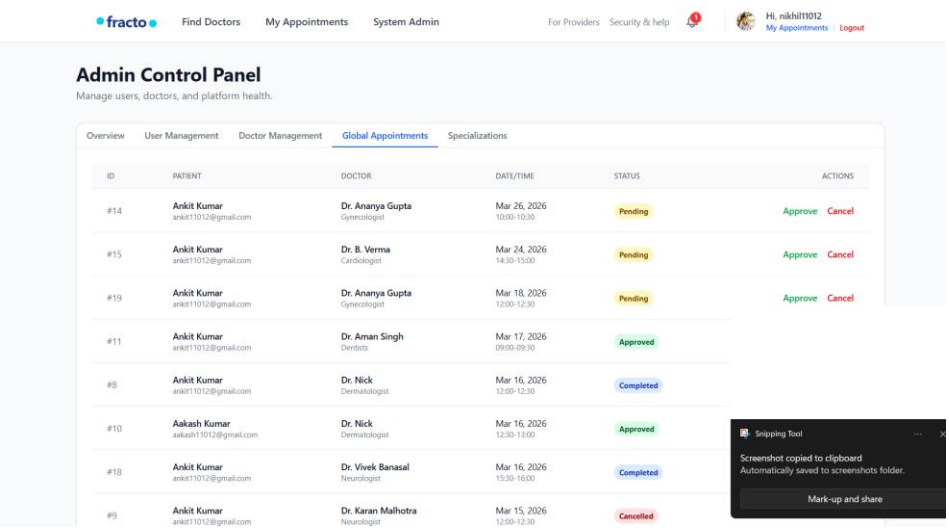




Notification Triggers:

User Action	Who Gets Notified	Message
User books appointment	All Admins	"{PatientName} has booked an appointment with Dr. {DoctorName}. 📅"
Admin approves appointment	Specific User	"Your appointment has been approved! ✅"
Admin marks completed	Specific User	"Your appointment is complete! You can now rate your doctor. ☆"
Admin cancels appointment	Specific User	"Your appointment has been cancelled by the administrator. ✕"

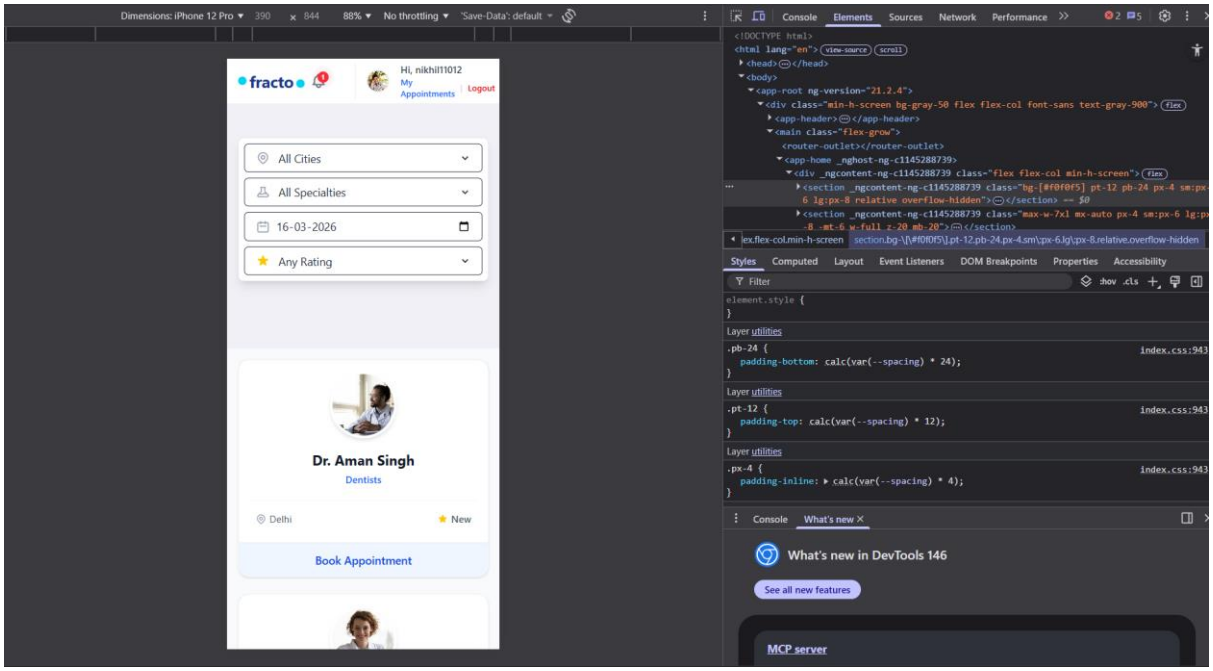


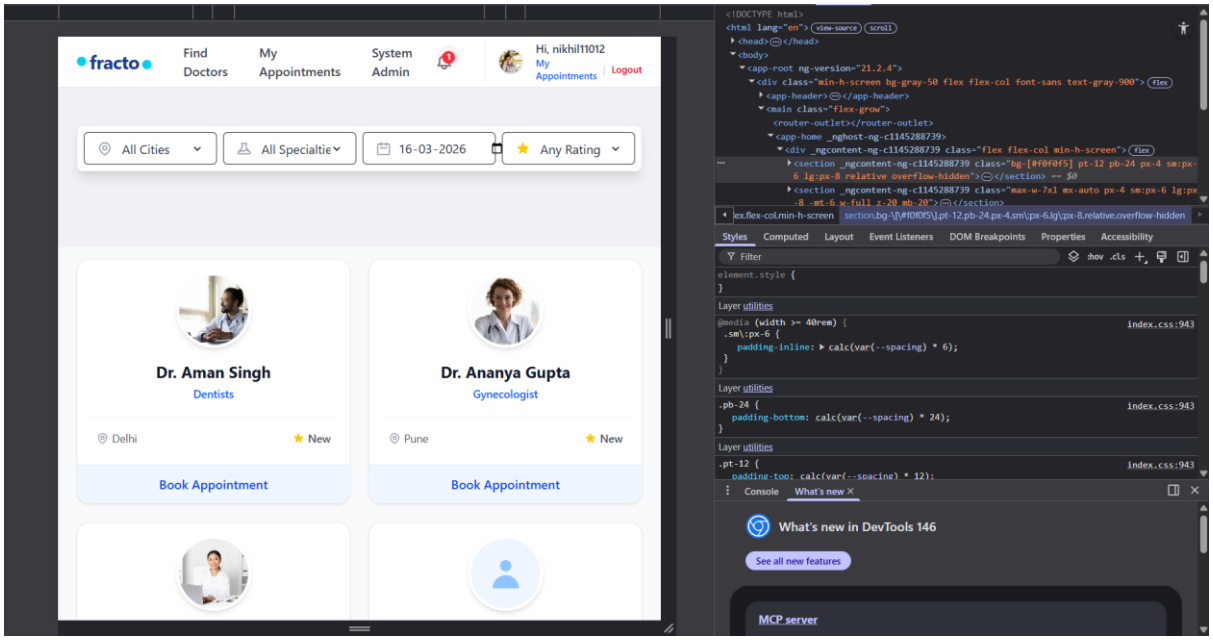


Responsiveness (Page-wise, Tailwind CSS)

- **Global layout (Header + Footer + Content)**
  - The app uses a responsive container pattern (max-w-7xl mx-auto px-4 sm:px-6 lg:px-8) so content has proper margins on all screens.
  - Header is **sticky** (sticky top-0) and stays accessible on scroll, while footer uses a responsive grid.
- **Header / Navigation (Header)**
  - Desktop navigation is shown only on medium+ screens using hidden md:flex.
  - On mobile, the layout stays compact with flex alignment (flex justify-between items-center h-16) and spacing utilities (gap-*) so it doesn't overflow.
- **Home / Doctor Search (Home)**
  - Search filters use a responsive grid: grid grid-cols-1 md:grid-cols-4, so filters stack in **1 column on mobile** and become **4 columns on desktop**.
  - Doctor listing uses responsive columns: grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 xl:grid-cols-4, scaling from **1 → 2 → 3 → 4 columns** based on viewport size.
  - The "Specialties" section uses a responsive flex row that supports horizontal scrolling on small screens (flex overflow-x-auto) so items don't shrink unreadably.
- **Login Page (Login)**
  - The main layout switches from stacked to split view using flex flex-col md:flex-row.
  - The illustration section is hidden on small screens (hidden md:flex) to keep the login form centered and readable.
  - The form container uses w-full md:w-1/2 max-w-md so it remains full width on mobile but doesn't stretch too wide on desktop.
- **Register Page (Register)**
  - Same responsive pattern as Login: flex flex-col md:flex-row with the illustration hidden on mobile (hidden md:flex).
  - Form stays readable using w-full md:w-1/2 max-w-md and consistent spacing (space-y-6, px-4 sm:px-6 lg:px-8).
- **Doctor Details + Booking (Doctor Details)**

    - Doctor details card uses flex flex-col md:flex-row so the image and info stack on mobile and align side-by-side on larger screens.
    - Booking time slots use a responsive grid: grid grid-cols-3 sm:grid-cols-4 md:grid-cols-5, increasing available columns as screen size grows.
- **Booking Confirmation (Booking Confirmation)**
    - Uses a centered responsive container (max-w-md w-full) with padding utilities (px-4 sm:px-6 lg:px-8) to maintain consistent spacing across devices.
    - Action buttons stack vertically (flex flex-col space-y-3) for easy tapping on mobile.
- **My Appointments (My Appointments)**
    - Appointment cards use responsive grid columns: grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3, showing **1 column on mobile** and scaling up on larger screens.
    - Each card uses flexible layout (flex flex-col, flex-grow) to keep button sections aligned even when text wraps.
- **Admin Dashboard (Admin Dashboard)**
    - Tabs use flex overflow-x-auto so the tab bar remains usable on smaller screens without breaking layout.
    - The “stats” tiles are responsive: grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4.
    - Data tables are wrapped in overflow-x-auto, enabling horizontal scroll on mobile rather than squeezing columns.
- **Footer (Footer)**
    - Footer links use a responsive grid: grid grid-cols-2 md:grid-cols-4 lg:grid-cols-6, scaling from **2 → 4 → 6 columns** based on viewport width.


fracto
Hi, nikhil11012
My Appointments
Logout
All Cities
All Specialties
16-03-2026
Any Rating
Dr. Aman Singh
Dentists
Delhi
New
Book Appointment
Console
Elements
Sources
Network
Performance
Styles
Computed
Layer utilities
.pb-24 {
padding-bottom: calc(var(--spacing) * 24);
}
.pt-12 {
padding-top: calc(var(--spacing) * 12);
}
.px-4 {
padding-inline: calc(var(--spacing) * 4);
}
What's new in DevTools 146
See all new features
MCP server


fracto
Find Doctors
My Appointments
System Admin
Hi, nikhil11012
My Appointments
Logout
All Cities
All Specialtie
16-03-2026
Any Rating
Dr. Aman Singh
Dentists
Delhi
New
Book Appointment
Dr. Ananya Gupta
Gynecologist
Pune
New
Book Appointment
Styles
Computed
Layer utilities
@media (width >= 40rem) {
.sm\:px-6 {
padding-inline: calc(var(--spacing) * 6);
}
}
.pb-24 {
padding-bottom: calc(var(--spacing) * 24);
}
What's new in DevTools 146
See all new features
MCP server

11. Session Handling

Session management is handled entirely by the AuthService

How Session is Stored

```
// auth.service.ts

private readonly TOKEN_KEY = 'fracto_token';
private readonly ROLE_KEY  = 'fracto_role';

private setSession(authResult: AuthResponse) {
    // Store JWT token and user role in sessionStorage
    // sessionStorage is cleared automatically when the browser tab is closed
    sessionStorage.setItem(this.TOKEN_KEY, authResult.token);
    sessionStorage.setItem(this.ROLE_KEY, authResult.role.toString());

    // Update reactive signal so UI reflects login state immediately
    this.currentUser.set({
        username:         authResult.username,
        role:             authResult.role,
        profileImagePath: authResult.profileImagePath
    });
}
```

Session Restoration on Refresh

```
private restoreSession() {
    const token = this.getToken();
    const role  = sessionStorage.getItem(this.ROLE_KEY);

    if (token && role) {
        // Set a placeholder to avoid UI flash of logged-out state
        this.currentUser.set({ username: 'Loading...', role: parseInt(role, 10) });

        // Verify token with backend using a bypass client (skips interceptor to avoid circular dependency)
        this.httpBackendClient.get<any>(`${environment.apiUrl}/Auth/me`, {
            headers: { Authorization: `Bearer ${token}` }
        }).subscribe({
            next: (user) => this.currentUser.set({ ... }),
            error: () => this.logout()  // Token expired — clear session
        });
    }
}
```

Logout

```
logout() {
    sessionStorage.removeItem(this.TOKEN_KEY);
    sessionStorage.removeItem(this.ROLE_KEY);
    this.currentUser.set(null);       // Clear reactive state
    this.router.navigate(['/login']); // Redirect to login page
}
```

Why session Storage instead of localStorage?

sessionStorage is automatically cleared when the browser tab is closed, providing a natural session boundary. This prevents stale tokens from persisting across different sessions on a shared computer, making it a more secure default.

12. Angular – Backend Integration & JWT Security

Backend JWT Configuration (appsettings.json)

```
{
  "Jwt": {
    "Key":            "REPLACE_WITH_A_LONG_RANDOM_SECRET_KEY_32+CHARS",
    "Issuer":         "Fracto",
    "Audience":       "FractoClient",
    "ExpiryMinutes": 60
  }
}
```

Token Validation Parameters (Program.cs)

```
options.TokenValidationParameters = new TokenValidationParameters
{
    ValidateIssuer           = true,
    ValidateAudience         = true,
    ValidateLifetime         = true,          // Rejects expired tokens
    ValidateIssuerSigningKey = true,          // Verifies the signature
    ValidIssuer              = jwt.Issuer,    // "Fracto"
    ValidAudience            = jwt.Audience,  // "FractoClient"
    IssuerSigningKey         = new SymmetricSecurityKey(
                                   Encoding.UTF8.GetBytes(jwt.Key)),
    ClockSkew                = TimeSpan.FromSeconds(30), // 30s tolerance
    NameClaimType            = ClaimTypes.Name,
    RoleClaimType            = ClaimTypes.Role
};
```

CORS Configuration

```
builder.Services.AddCors(options =>
{
    options.AddPolicy("FractoFrontend", policy =>
    {
        policy.WithOrigins("http://localhost:4200", "http://localhost:4900")
              .AllowAnyHeader()
              .AllowAnyMethod()
              .AllowCredentials();  // Required for SignalR WebSocket upgrade
    });
});
```

End-to-End Flow: User Login to Protected API Call

1. User types credentials → Angular Login Component

2. LoginComponent calls authService.login()

3. authService sends POST /api/auth/login with { identifier, password }

4. Backend verifies credentials, returns JWT token

5. authService.setSession() stores token in sessionStorage

6. User navigates to /my-appointments (guarded by authGuard)

7. authGuard checks authService.isAuthenticated() → true → allows navigation

8. MyAppointments component calls appointmentService.getMyAppointments()

9. authInterceptor reads token from sessionStorage, adds Authorization header

10. Backend validates JWT → extracts UserId from claims → returns only that user's appointments

13. Testing - Component & End-to-End

Unit Test — Root App Component (app.spec.ts)

The Angular project includes an initial unit test for the root App component using the Angular Testing Utilities:

```
// app/app.spec.ts
import { TestBed } from '@angular/core/testing';
import { App } from './app';

describe('App', () => {
  beforeEach(async () => {
    await TestBed.configureTestingModule({
      imports: [App],  // Standalone component
    }).compileComponents();
  });

  it('should create the app', () => {
    const fixture = TestBed.createComponent(App);
    const app     = fixture.componentInstance;
    expect(app).toBeTruthy();  // App instantiates without errors
  });

  it(`should have the 'Fracto.Web' title`, () => {
    const fixture = TestBed.createComponent(App);
    const app     = fixture.componentInstance;
    expect(app.title).toEqual('Fracto.Web');
  });

  it('should render title', () => {
    const fixture = TestBed.createComponent(App);
    fixture.detectChanges();
    const compiled = fixture.nativeElement as HTMLElement;
    expect(compiled.querySelector('h1')?.textContent).toContain('Hello, Fracto.Web');
  });
});
```

How to Run Tests

```
# Navigate to the frontend directory
cd Fracto.Web

# Run all unit tests (uses Karma test runner + Jasmine)
ng test

# Run with code coverage report
ng test --code-coverage
```

Testing Tools Used

Tool	Purpose
Karma	Test runner (launches Chrome headless)
Jasmine	Testing framework (describe/it/expect syntax)
TestBed	Angular's testing module for component instantiation

14. GitHub Repository & Version Control

- **Repository Type:** Public GitHub Repository
- **Backend Project:** Fracto.Api — ASP.NET Core 8 Web API
- **Frontend Project:** Fracto.Web — Angular 18

Repository Structure

```
Fracto.Api/              ← Solution root
├── Fracto.Api/          ← Backend Web API project
│   ├── Controllers/     ← 11 API controller files
│   ├── Models/          ← 5 entity model files
│   ├── DTOs/            ← Data Transfer Object folders
│   ├── Data/            ← AppDbContext (EF Core)
│   ├── Hubs/            ← SignalR NotificationHub
│   ├── Migrations/      ← EF Core database migrations
│   ├── Program.cs       ← Application entry point & DI config
│   └── appsettings.json ← Connection string & JWT config
└── Fracto.Web/          ← Angular frontend project
    ├── src/app/core/    ← Shared services, guards, interceptors
    ├── src/app/features/← Feature modules (auth, admin, doctors)
    └── angular.json     ← Angular workspace config
```